

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	CH.POORNA TEJA	Reg. No.:	192472149
Section / Batch:	2023	Date:	29/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Q47. Debugging String Matching Code (Incorrect Handling of Empty Pattern Case)

The following brute force string matching function does not correctly handle the special case where the pattern is empty, which can cause ambiguous or unintended output. Carefully analyze the behavior of the loop structure for this edge case and explain why explicit handling is needed. Debug the code so that empty-pattern input is treated consistently and correctly. Optimize the implementation for robustness and readability without changing the brute force strategy for normal cases. Analyze the time complexity of the corrected version and rewrite the final code with proper documentation.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    matches = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        else:
            matches.append(i)
    return matches
```

Code of Conduct

I CH.POORNA TEJA(192472149)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ch.poorna teja_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q47. Debugging String Matching Code (Incorrect Handling of Empty Pattern Case)

Problem Interpretation

The given brute-force string matching function searches for all occurrences of a pattern within a text by comparing characters one by one. However, it does not explicitly handle the special case where the pattern is empty. An empty pattern is an important edge case because it can lead to ambiguous or unintended results. Therefore, the code should explicitly check for this condition before performing the normal brute-force search. This improves correctness, robustness, and readability while preserving the brute-force strategy for all normal cases.

Original Code

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    matches = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        else:
            matches.append(i)
    return matches
```

Identifying the Issue

The code does not explicitly handle the case when the pattern is empty.

When `pattern = ""`,

- `m = 0`
- The inner loop becomes:

```
for j in range(0):
```

Since `range(0)` executes zero iterations, the loop body is skipped completely.

The for-else block immediately executes the else clause, causing every position in the text to be added to the matches list.

Example:

```
text = "HELLO"
pattern = ""
```

Output produced:

```
[0, 1, 2, 3, 4, 5]
```

Although some programming languages define an empty pattern as matching every position, this behavior is not obvious from the code and may be confusing or unintended in many applications. Therefore, explicit handling is recommended.

Why Explicit Handling Is Needed

Without an explicit condition:

- The algorithm relies on Python's for-else behavior.
- The result may appear unexpected to users.
- The special case is hidden inside the loop logic, reducing readability.
- Future maintenance becomes difficult because the behavior is not immediately clear.

Adding an explicit check makes the implementation easier to understand and ensures consistent behavior.

Step-by-Step Debugging

Fix 1:

Check whether the pattern is empty before starting the brute-force search.

Fix 2:

If the pattern is empty, return an empty list (or another clearly defined result depending on application requirements).

Fix 3:

Keep the original brute-force comparison unchanged for all normal cases.

Corrected and Optimized Code

```
```python
def string_match(text, pattern):
 """
 Finds all occurrences of a pattern in a text using the
 Brute Force String Matching algorithm.
```

Fixes applied:

- Explicitly handles the empty-pattern case.
- Preserves the original brute-force approach.
- Improves readability and robustness.

Parameters:

text (str): Input text.  
pattern (str): Pattern to search.

Returns:

list: Starting indices where the pattern occurs.

```
"""
```

```
Edge case: empty pattern
```

```
if len(pattern) == 0:
 return []
```

```
n = len(text)
```

```
m = len(pattern)
```

```

matches = []

for i in range(n - m + 1):
 for j in range(m):
 if text[i + j] != pattern[j]:
 break
 else:
 matches.append(i)

return matches
```

```

Verification of Correctness

Test 1:

```

text = "ABABCABC"
pattern = "ABC"

```

Output:

[2, 5]

Correct ✓

Test 2:

```

text = "AAAAA"
pattern = "AA"

```

Output:

[0, 1, 2, 3]

Correct ✓

Test 3:

```

text = "HELLO"
pattern = ""

```

Output:

[]

Correct ✓

Test 4:

```

text = ""

```

```
pattern = "ABC"
```

Output:

```
[]
```

Correct ✓

Test 5:

```
text = ""  
pattern = ""
```

Output:

```
[]
```

Correct ✓

The corrected implementation now handles both normal inputs and edge cases consistently.

Time Complexity Analysis

| Case | Description | Complexity |
|--------------|--|-----------------|
| Best Case | First character mismatches at every position | $O(n)$ |
| Average Case | Partial comparisons before mismatch | $O(n \times m)$ |
| Worst Case | Every character compared at every position | $O(n \times m)$ |

Space Complexity

Only the matches list and a few variables are used.

Space Complexity = $O(1)$ (excluding the output list)

Summary of Fixes

| Aspect | Buggy Code | Corrected Code |
|----------------------|---------------------------------|--------------------------------|
| Empty pattern | No explicit handling | Returns empty list immediately |
| Readability | Hidden behavior inside for-else | Explicit edge-case handling |
| Robustness | Ambiguous behavior | Predictable and consistent |
| Brute-force strategy | Preserved | Preserved |

Interpretation

This debugging exercise demonstrates the importance of handling edge cases explicitly. Although the original implementation works correctly for normal string matching, the empty-pattern case depends on Python's for-else behavior, which may produce confusing results. By adding a simple condition before the search begins, the algorithm becomes easier to understand, more reliable, and more maintainable while preserving the original brute-force strategy. The corrected algorithm still performs brute-force matching for all normal inputs with $O(n \times m)$ time complexity and provides predictable behavior for every possible input.